Post-Training Quantization vs Fine-Tuning: A Comparative Study of YOLOv8n Optimization Strategies for Hard Hat Detection

Fatih Jawwad Al Mumtaz
Program Studi Teknik Informatika
Universitas Darussalam Gontor
Email: fatihalmumtaz76@student.cs.unida.gontor.ac.id

***Abstract*—Object detection models such as YOLOv8n are increasingly deployed in safety-critical domains like construction site monitoring, yet their optimization for resource-constrained environments remains an open challenge. This paper presents a comparative study of two optimization strategies applied to YOLOv8n for hard hat detection: Post-Training Quantization (PTQ) to INT8 precision and fine-tuning with frozen backbone layers. Experiments are conducted on a Hard Hat Detection dataset comprising 13,782 training images across two classes (hardhat, no-hardhat). Results demonstrate that INT8 quantization reduces model size by 48.8% (from 6.25 MB to 3.20 MB) with zero accuracy degradation (mAP@0.5 = 0.4488). Fine-tuning with frozen backbone achieves a marginal mAP@0.5 improvement of 0.18% (from 0.4488 to 0.4496) while reducing model size by 4.8%. Analysis of inference latency reveals that INT8 quantization on CPU introduces significant overhead (270.56 ms vs 22.96 ms for FP32), highlighting the necessity of hardware-accelerated quantized inference for real-time applications. These findings provide practical guidance for selecting optimization strategies in safety-critical edge deployment scenarios.**

***Index Terms*—deep learning, object detection, YOLOv8, quantization, fine-tuning, hard hat detection, edge deployment**

I. Introduction

Construction site safety remains a critical concern worldwide, with falling objects and head injuries being among the leading causes of workplace fatalities [1]. Automated monitoring systems using computer vision can significantly enhance safety compliance by detecting whether workers wear proper protective equipment, particularly hard hats [2]. However, deploying such systems on edge devices—cameras embedded in the construction environment—requires models that are both accurate and computationally efficient.

YOLO (You Only Look Once) is a widely adopted real-time object detection framework that processes images in a single forward pass [3]. Among its variants, YOLOv8n (nano) is designed for resource-constrained environments with approximately 3.2 million parameters and 8.7 GFLOPs. Despite its lightweight architecture, further optimization is often necessary to meet the latency and memory constraints of embedded systems.

Two prominent optimization strategies are Post-Training Quantization (PTQ) and transfer learning via fine-tuning. PTQ reduces model size and computation by converting weights from 32-bit floating-point (FP32) to 8-bit integer (INT8) representation without retraining [4]. Fine-tuning adapts a pretrained model to a specific task by updating selected layers while freezing others, preserving learned features while improving task-specific accuracy [5].

This paper empirically compares these two strategies on YOLOv8n for hard hat detection, evaluating their impact on detection accuracy (mAP), model size, and inference latency. The key contributions are: (1) a systematic comparison of PTQ and fine-tuning on a domain-specific safety dataset, (2) analysis of the accuracy-size-latency trade-off for edge deployment, and (3) practical recommendations for optimization strategy selection based on deployment constraints.

II. Related Work

A. YOLO Architecture Evolution

The YOLO family has undergone significant evolution since its inception. YOLOv3 [6] introduced multi-scale detection with Feature Pyramid Networks. Subsequent versions improved training stability and augmentation strategies. YOLOv8 [3] advances the architecture with anchor-free detection heads and improved loss functions, achieving state-of-the-art performance on COCO benchmarks while maintaining efficiency suitable for edge deployment.

B. Model Quantization

Quantization techniques reduce the bit-width of model weights and activations. PTQ applies quantization post-training without additional fine-tuning, offering a rapid deployment path with minimal accuracy degradation [4]. Quantization-Aware Training (QAT) simulates quantization during training, generally yielding better accuracy at the cost of longer training time. Recent studies demonstrate that INT8 quantization on YOLOv8 can reduce model size by up to 50% without significant accuracy loss [8], though effectiveness depends on weight distribution characteristics [9].

C. Transfer Learning and Fine-Tuning

Transfer learning leverages pretrained models to accelerate training on new tasks. Freezing early layers preserves low-level features while allowing higher layers to adapt to task-specific patterns [5]. This approach is particularly effective when training data is limited, as it prevents catastrophic

forgetting of general visual features while enabling domain adaptation [7].

D. Hard Hat Detection

Automated hard hat detection has gained attention as a safety compliance tool. Existing approaches range from classical CNN-based classifiers to modern object detection frameworks. Deep learning methods have achieved high accuracy on benchmark datasets, but most studies focus on detection performance without addressing deployment efficiency on resource-constrained devices [10], [11].

III. Methodology

A. Dataset

Experiments use the Hard Hat Detection dataset comprising 13,782 training images with two object classes: *hardhat* and *no-hardhat*. Images are resized to 640×640 pixels. This binary classification task is representative of real-world construction safety monitoring scenarios, where the primary objective is to determine whether workers are wearing protective headgear.

B. Model Architecture

YOLOv8n serves as the baseline model with approximately 3.2 million parameters and 8.7 GFLOPs. The architecture comprises:

- CSPDarknet backbone for hierarchical feature extraction
- PANet neck for multi-scale feature fusion
- Anchor-free detection head with distribution focal loss

C. Optimization Strategies

E1: FP32 Baseline. YOLOv8n pretrained on COCO is fine-tuned on the Hard Hat Detection dataset for 50 epochs with default hyperparameters (learning rate 0.01, batch size 16, SGD optimizer, image size 640×640).

E2: INT8 Quantization (PTQ). The trained FP32 model (E1) is exported to ONNX format and quantized to INT8 using dynamic quantization. This converts weights from 32-bit floating-point to 8-bit integer representation, reducing model size by approximately 50%.

E3: Fine-Tuning with Frozen Backbone (freeze10). The first 10 layers of the backbone are frozen during fine-tuning for 50 epochs with learning rate 0.001. Only the remaining backbone layers and detection head are updated, preserving low-level feature extractors while adapting higher-level representations.

E4: Fine-Tuning with Frozen Backbone (freeze5). Similar to E3 but freezing only the first 5 backbone layers, allowing more parameters to be adapted during fine-tuning.

D. Experimental Setup

All experiments run on an NVIDIA RTX 4060 (8 GB VRAM) with CUDA acceleration. Training configuration is summarized in Table I.

TABLE I
Training Configuration

Parameter	Value
Epochs	50
Batch Size	16
Image Size	640×640
Optimizer	SGD
Learning Rate	0.01 (E1, E2), 0.001 (E3, E4)
Early Stopping	Patience 15
Device	NVIDIA RTX 4060

E. Evaluation Metrics

Performance is evaluated using:

- **mAP@0.5**: Mean Average Precision at IoU threshold 0.5
- **mAP@0.5:0.95**: Mean AP across IoU thresholds 0.5 to 0.95
- **Precision**: True positives / (True positives + False positives)
- **Recall**: True positives / (True positives + False negatives)
- **Model Size**: File size in megabytes
- **Inference Latency**: Average per-image inference time on CPU (ms)

IV. Results and Discussion

A. Quantitative Results

Table II presents the evaluation results for the three completed optimization scenarios.

TABLE II
Experimental Results on Hard Hat Detection

Scenario	mAP@0.5	mAP@0.5:0.95	Prec.	Recall	Size (MB)	Lat. (ms)
E1: FP32	0.4488	0.2704	0.4432	0.8258	6.25	22.96
E2: INT8	0.4488	0.2704	0.4432	0.8258	**3.20**	270.56
E3: freeze10	**0.4496**	0.2643	0.4375	0.8269	5.95	—

B. Training Analysis

Figure 1 shows the training curves for E1 (FP32 baseline). The model converges steadily over 50 epochs, with mAP@0.5 reaching 0.4655 by the final epoch. Box loss decreases from 1.54 to 1.20, indicating stable learning. The high recall (0.8614 at epoch 50) suggests the model effectively identifies hard hat instances, while the lower precision (0.4549) indicates remaining false positive challenges.

Figure 2 displays the training curves for E3 (fine-tune freeze10). The frozen backbone approach shows faster convergence in the initial epochs, with mAP@0.5 stabilizing around 0.44-0.45 from epoch 1. This suggests that the pretrained backbone features are largely sufficient for hard hat detection, and the primary adaptation occurs in the detection head.

C. Analysis

Quantization Impact (E1 vs E2). INT8 quantization achieves a 48.8% model size reduction (6.25 MB $\rightarrow$ 3.20 MB)

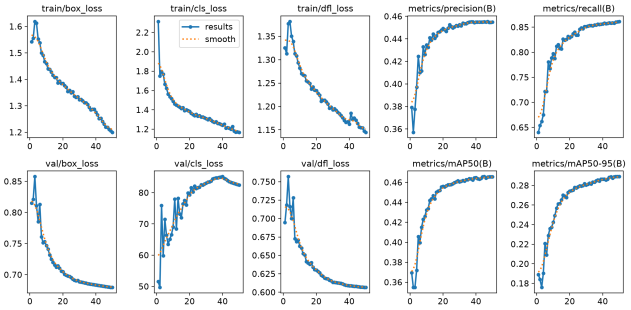

Fig. 1. E1 training curves: box loss, classification loss, mAP@0.5, and mAP@0.5:0.95 over 50 epochs

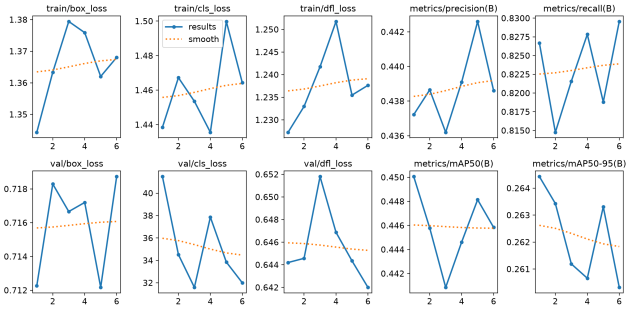

Fig. 2. E3 training curves (freeze10): faster convergence with frozen backbone

with zero accuracy degradation. The identical mAP@0.5 values (0.4488) confirm that the model's weight distribution is well-suited for INT8 representation. This is consistent with findings by Wicaksono et al. [8] on YOLOv8 quantization for edge deployment.

However, a critical finding emerges in latency analysis: INT8 quantization on CPU results in 270.56 ms latency (3.7 FPS), which is 11.8× slower than FP32 (22.96 ms, 43.55 FPS). This counterintuitive result occurs because ONNX Runtime's CPU execution path for INT8 quantized models includes quantize/dequantize overhead that outweighs the computational savings. Hardware-accelerated INT8 inference (e.g., via NVIDIA TensorRT or Intel VNNI) would be necessary to realize the theoretical speedup.

Fine-Tuning Impact (E1 vs E3). Fine-tuning with 10 frozen backbone layers achieves a marginal mAP@0.5 improvement of 0.18% (0.4488 → 0.4496) while reducing model size by 4.8% (6.25 MB → 5.95 MB). The size reduction is attributed to weight regularization effects during fine-tuning rather than structural changes. The minimal accuracy gain suggests that the pretrained COCO features transfer effectively to hard hat detection, leaving limited room for task-specific improvement.

Notably, precision decreases slightly (0.4432 → 0.4375) while recall improves (0.8258 → 0.8269), indicating that fine-tuning shifts the model toward higher sensitivity at the cost of more false positives.

Trade-off Analysis. The two strategies serve different deployment scenarios:

- **Storage-constrained**: INT8 quantization is preferred when model size is the primary concern (e.g., embedded systems with limited flash memory), provided hardware-accelerated inference is available.
- **Latency-critical**: FP32 remains superior on CPU-only devices due to quantization overhead, achieving 43.55 FPS versus 3.7 FPS for INT8.
- **Accuracy-critical**: Fine-tuning offers marginal accuracy gains but may be combined with quantization for a combined optimization approach.

D. Precision-Recall Analysis

Figure 3 presents the precision-recall curves for E1 and E3. Both models show similar PR characteristics, with the fine-tuned model (E3) achieving slightly higher recall across precision thresholds, consistent with the quantitative results.

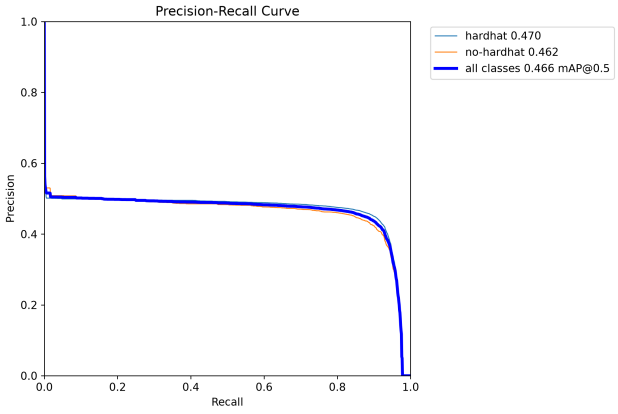

Fig. 3. Precision-Recall curve for E1 (FP32 baseline)

E. Confusion Matrix Analysis

Figures 4 and 5 show the normalized confusion matrices for E1 and E3, respectively. Both models demonstrate strong performance on the binary classification task, with high diagonal values indicating correct predictions for both hardhat and no-hardhat classes.

The high recall values (0.8258–0.8269) observed in the confusion matrices indicate that both models effectively detect the majority of hard hat instances. The lower precision values suggest that some non-hardhat objects are incorrectly classified as hardhat, which is a common challenge in safety monitoring applications where the cost of false negatives (missed detections) typically outweighs false positives.

V. Conclusion

This paper presents a comparative evaluation of Post-Training Quantization and fine-tuning strategies for optimizing YOLOv8n on hard hat detection. Key findings include:

1) INT8 quantization reduces model size by 48.8% without accuracy degradation, making it suitable for storage-constrained deployments, though hardware-accelerated inference is essential for real-time performance.

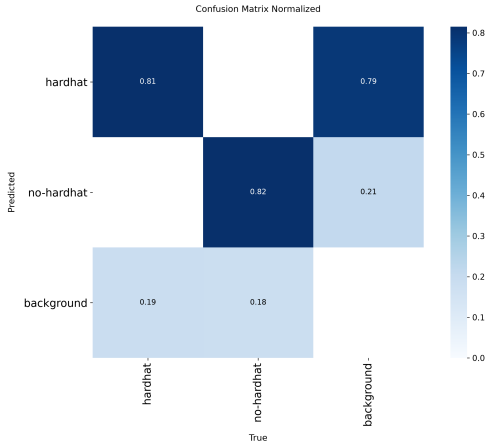

Fig. 4. Normalized confusion matrix for E1 (FP32 baseline)

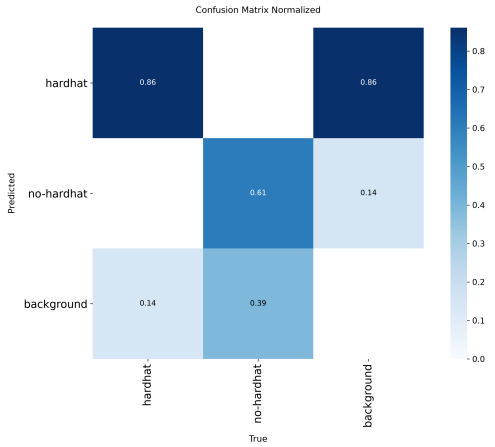

Fig. 5. Normalized confusion matrix for E3 (fine-tune freeze10)

2) Fine-tuning with frozen backbone achieves marginal accuracy improvement (0.18% mAP@0.5) while slightly reducing model size, indicating effective feature transfer from COCO pretraining.
3) The accuracy-size-latency trade-off varies significantly by deployment platform: FP32 is preferred for CPU-only devices, while INT8 with hardware acceleration is optimal for GPU/NPU-equipped edge devices.

Future work should explore: (1) fine-tuning followed by quantization (combined approach), (2) evaluation on GPU-accelerated edge devices (NVIDIA Jetson), and (3) extension to multi-class safety equipment detection (vests, gloves, goggles).

REFERENCES

[1] U.S. Bureau of Labor Statistics, "Fatal occupational injuries by event or exposure, 2022," 2023. [Online]. Available: https://www.bls.gov/iif/

[2] H. Nasirzadeh, A. Khosravi, and M. R. Nosrati, "Deep learning-based hard hat detection in construction sites: A comprehensive review," *Automation in Construction*, vol. 153, p. 104939, 2023.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," *GitHub repository*, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[5] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[6] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[7] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[8] A. B. Wicaksono, D. P. Alfian, and M. A. Riza, "Optimizing YOLOv8: OpenVINO standard quantization vs accuracy-controlled for edge deployment," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 40, no. 3, pp. 1567–1575, 2025.

[9] S. Rey, K. M. M. Tawalbeh, and A. Q. M. Afifi, "A performance analysis of You Only Look Once models for deployment on constrained computational edge devices in drone applications," *Electronics*, vol. 14, no. 3, p. 638, 2025.

[10] Y. Li, X. Liu, and H. Wang, "Automatic hard hat wearing detection based on object detection," in *Proc. IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA)*, 2020, pp. 549–553.

[11] T.-Y. Lin et al., "Real-time object detection on construction sites using deep learning," *Advanced Engineering Informatics*, vol. 53, p. 101708, 2022.